Bayesian Network Reconstruction of Interbank Exposures (Nepal)

Krishna Neupane

2026-10-23

Abstract

text text

1 Paper 1 — Bayesian Network Reconstruction of Interbank Exposures (Nepal)

Status: core algorithms (capacitated projection, synthetic generator, amortized training step) implemented and unit-tested; first end-to-end synthetic validation experiment (§6/§12) running and under active iteration — not yet a final result. **Target venue:** ICLR / NeurIPS / ICAIF, 7 pages.

One-line description: Reconstruct the latent bilateral interbank exposure matrix W_t for Nepali BFIs from NRB bulletin data, using an SBM-structured, covariate-informed generative model with a differentiable regulatory-constraint projection layer, amortized by a GNN/GRU inference network.

This supersedes the earlier version of `feature_model_mapping.md`, which had drifted from the actual design (it invented a generic "Bayesian-net structural-constraint" layer built from accounting identities — that was never part of the real design and is corrected below).

1.1 0. Problem framing and contribution to policy

Problem framing (what "the network" means here). We do not pre-specify the edges of the interbank network. Instead, following the network reconstruction literature (e.g., Squartini et al., 2018), we treat the bilateral exposure structure as latent and infer it from publicly observable aggregates — balance-sheet marginals and bank-level covariates — within a constrained statistical framework. Concretely: a node is a bank; a directed edge $i \to j$ is $W_{ij,t}$, the exposure of bank i to bank j; an edge "exists" when $W_{ij,t} > 0$. Neither the edge set nor the weights are observed — only the row/column marginals and covariates are. This is a problem of statistical inference under partial observability, not analysis over a topology that is already known or self-reported. (Revisit/tighten this paragraph later — current form is a first pass.)

Contribution to policy. No central bank in a resource-constrained setting is going to mandate new bilateral exposure reporting purely for a research exercise — that is a costly, politically slow regulatory

change. The contribution here is that this doesn't require that change: a usable, continuously-updatable estimate of systemic interconnectedness is produced from data NRB (or any similarly-resourced central bank) already publishes every month, with zero new reporting burden. Put to a central banker directly: today there is no way to ask "who is exposed to whom" or "which banks are structurally systemic" — not because the question doesn't matter, but because answering it was assumed to require data that isn't collected. This method shows it doesn't. It converts a previously unanswerable supervisory question into an answerable one, cheaply, and produces a reusable pipeline that other analyses (stress tests, contagion simulations, Paper 2's spillover estimation) can plug into. This also generalizes beyond Nepal: most emerging-market central banks publish the same kind of aggregate call-report data and face the identical constraint, so the method is a template for resource-constrained regulators broadly, not a Nepal-specific workaround.

1.2 1. Why this is its own paper

Separated from the causal/policy question (Paper 2) because the two are methodologically unrelated: this paper's contribution is a reconstruction method with no causal-identification content; Paper 2's contribution is a causal-identification design that *consumes* this paper's output as a fixed input. Bundling them would force a methods paper to carry a policy narrative it doesn't need, and vice versa.

1.3 2. Notation

Symbol	Meaning
$i, j \in \{1, ..., N\}$	institutions (BFIs)
$t \in \{1, ..., T\}$	periods (NRB bulletin months)
$X_{i,t} \in \mathbb{R}^d$	node covariates for bank i at time t
$r_{i,t}, c_{i,t} \geq 0$	observed interbank lending / borrowing marginals
$K_{i,t}$	core capital (used to set the regulatory single-obligor cap)
$W_t \in \mathbb{R}_{\geq 0}^{N \times N}$	**latent** bilateral exposure matrix — never observed, the reconstruction target
$z_i \in \{1, ..., K\}$	latent block/tier membership (e.g. core vs. periphery)

1.4 3. Generative model

- Block membership: $z_i \sim \text{Categorical}(\pi)$ — (Eq. 1)
- Block-pair intensity: $\Lambda \in \mathbb{R}_{\geq 0}^{K \times K}$ — the SBM core
- Fitness/degree-correction from covariates: $\phi_{i,t} = \exp(w^\top X_{i,t})$ — (Eq. 2)

- Latent edge intensity:

$$\lambda_{ij,t} = \Lambda_{z_i z_j} \cdot \phi_{i,t}\phi_{j,t} \cdot \exp(h_{i,t}^\top h_{j,t}) \qquad \text{(Eq. 3)}$$

- Zero-inflated edge weight (most bank pairs have *no* exposure):

$$\Pr(\tilde{W}_{ij,t} = 0) = \sigma(-\eta_{ij,t}), \qquad \tilde{W}_{ij,t} \mid \tilde{W}_{ij,t} > 0 \sim \text{LogNormal}(\log \lambda_{ij,t}, \sigma^2) \qquad \text{(Eq. 4)}$$

- Marginals are a **deterministic function** of W_t:

$$r_{i,t} = \sum_j W_{ij,t}, \qquad c_{j,t} = \sum_i W_{ij,t} \qquad \text{(Eq. 5)}$$

  This means the marginal constraint belongs in the *architecture* (a hard projection), not the likelihood — see §5.

1.5 4. Inference network (amortized, PyG GNN + GRU)

The encoder runs on covariates/history only — it cannot message-pass over W_t itself, since that would be circular (the whole point is that W_t is unobserved).

- Temporal encoder: $h_{i,t} = \text{GRU}_\theta([X_{i,t}; r_{i,t}; c_{i,t}], h_{i,t-1})$ (Eq. 6), optionally refined by a GNN layer over an auxiliary *observed* graph if one exists (e.g. common-sector exposure — see §8). $r_{i,t}, c_{i,t}$ (observed interbank lending/borrowing marginals — real, measured aggregate fields, not derived from the model's own inference of the latent W_t, so this doesn't violate the no-message-passing-over-W_t rule above) were added to the encoder's input after the first synthetic validation run showed block recovery is *impossible* from X alone: z is drawn independently of X (Eq. 1 vs. Eq. 2), so $q_\psi(z)$ conditioned on X-history only has no signal to learn from, no matter how the KL term in Eq. 9 is weighted (§10).
- Amortized block-assignment posterior: $q_\psi(z_i \mid X_{i,1:t}, r_{i,1:t}, c_{i,1:t}) = \text{Categorical}(\text{softmax}(\text{MLP}_\psi(h_{i,t})))$ (Eq. 7)
- Pairwise decoder produces $(\eta_{ij,t}, \lambda_{ij,t})$ from $(h_{i,t}, h_{j,t}, z_i, z_j)$ — full amortized variational posterior $q_\psi(W_t \mid X_{1:t}, r_t, c_t)$
- Reparameterized sampling: log-normal reparam trick for the continuous weight, Gumbel-softmax for the discrete existence gate.

1.6 5. Constraint-satisfaction layer (the technical core of the paper)

Differentiable **capacitated marginal projection**, via Dykstra's alternating-projections algorithm (an ADMM-family method) onto the intersection of three convex sets:

$$W_t = \text{Proj}_{\mathcal{R}\cap\mathcal{C}\cap\mathcal{B}}(\tilde{W}_t) \qquad \text{(Eq. 8)}$$

where $\mathcal{R} = \{W : \sum_j W_{ij} = r_{i,t}\}$, $\mathcal{C} = \{W : \sum_i W_{ij} = c_{j,t}\}$, $\mathcal{B} = \{W : 0 \le W_{ij} \le \kappa K_{i,t}\ \forall j\}$ —

i.e. the nearest-in-Euclidean- norm matrix to $\tilde{W}_t$ that satisfies the observed marginals and the regulatory single-obligor cap *exactly*, not approximately.

Revision note (superseded design): an earlier draft used a capacitated Sinkhorn/entropic-OT iteration with a `min(...)` row-scalar clip to enforce the cap. `test_sinkhorn.py`'s `binding_cap_case` showed this doesn't implement $\mathcal{B}$ at all — a single row-scalar u_i can only rescale an entire row uniformly, so "clip to satisfy the cap" collapsed into capping the *row sum* ($\sum_j W_{ij} \le \kappa K_{i,t}$) rather than *each entry* ($W_{ij} \le \kappa K_{i,t}\ \forall j$) — a materially different, and wrong, constraint (total interbank activity vs. concentration to any one counterparty). Plain (uncapped) Sinkhorn/IPF is still used, unmodified, as the classical Gandy-Veraart/RAS baseline in §7/§12 — that comparison never needed the cap in the first place, so it's unaffected by this fix.

References: Cuturi (2013) entropic OT (for the uncapped baseline); Boyle & Dykstra (1986) for the alternating-projection algorithm; Mena et al. (2018) Gumbel-Sinkhorn (unrelated to this layer — that's the discrete existence gate, Eq. 4). Flagged explicitly as "the one genuinely nontrivial implementation piece" in the design discussion — this is the part most likely to need real engineering effort, not the GNN/GRU scaffolding around it.

Correction vs. the earlier `feature_model_mapping.md`: there is no "Bayesian-net structural-constraint edge" tying `TOTAL Liabilities = TOTAL ASSETS` or `Net Profit` into the graph. That was never part of this design. The accounting identities in the raw NRB statements are a **data-quality cross-check** on the extractor (do the reported totals reconcile?), not a component of the generative model or the constraint layer.

1.7 6. Training objective

$$\mathcal{L} = -\mathbb{E}_q[\log p(W_t^{\text{true}})]_{\text{synthetic only}} + \beta_t \,\mathrm{KL}(q_\psi(z) \,\|\, p(z)) + \lambda_{\text{smooth}} \sum_t \|W_t - W_{t-1}\|_F^2 + \lambda_{\text{reg}} \|\Lambda, w\|^2$$

- β_t **(KL annealing weight)**: added after the first synthetic validation run showed the unweighted ($\beta_t = 1$ always) KL term collapsing $q_\psi(z)$ to the uniform prior almost immediately (§10) — the block-assignment posterior was getting worse than a trivial majority-guess baseline as training proceeded, even while the reconstruction likelihood kept improving via the decoder's direct use of h. β_t ramps linearly $0 \to \beta_{\max}$ over the first `KL_WARMUP_EPOCHS` epochs (standard β-VAE warm-up), giving $q_\psi(z)$ room to latch onto block signal before the KL penalty pulls it toward $p(z)$. $\beta_{\max} < 1$ **(not 1)**: after the Eq. 6-7 encoder fix (feeding r_t, c_t in) made real block recovery possible — block-recovery accuracy peaked at 0.85, well above the trivial baseline, while β_t was still small — annealing all the way to $\beta_t = 1$ reproduced the same collapse on a delay: accuracy decayed back toward/below the trivial baseline in lockstep with β_t's climb, well before warm-up even finished. So the fix wasn't ramp *speed*, it was the *ceiling*: $\beta_t = 1$ always eventually out-competes whatever reconstruction-gradient signal keeps $q_\psi(z)$ informative. Current value: $\beta_{\max} = 0.15$ (`KL_BETA_MAX` in the synthetic experiment script) — **partially validated, seed-dependent**: a single 3000-epoch run (seed 0) held block-recovery accuracy at 0.7578–0.7682

throughout (vs. that run's trivial baseline of 0.7344) rather than decaying back down, with KL stabilizing around 266 instead of collapsing to ~0. A 5-seed replication of that same run (seeds 0–4) showed this does **not** hold up as a general fix: block-recovery margin over the trivial baseline was $+0.0234, -0.1380, +0.1380, -0.1797, -0.0026$ (mean -0.0318, std 0.1148 — std nearly 4× the mean), with only 2/5 seeds beating the trivial baseline at all. Two distinct failure modes recur across seeds: full collapse to the uniform prior (KL ≈ 0 throughout, e.g. seed 4), and confident but *wrong* block assignment (KL climbs normally but accuracy sits below the trivial baseline, e.g. seed 1). Reconstruction accuracy, by contrast, **is** robust across the same 5 seeds: model mean relative error beat the classical Sinkhorn/RAS baseline in 5/5 seeds (mean 0.4154 vs. 0.5657). See §10 for what this means for the paper's claims.

- **Synthetic data** (known ground-truth W_t from an SBM generator): the first term is a real supervised reconstruction loss — this is the main validation evidence.
- **Real data** (no W_t is ever observed): the first term drops out entirely. What's left is a regularized/MAP-style constrained fit, *not* likelihood-based estimation. Evaluation on real data has to rely on indirect evidence: temporal stability of the recovered W_t sequence, and comparison against a classical baseline (§8).
- **Point estimate at inference/evaluation time**: training (Algorithm 3) samples z, e, W via the reparameterized Gumbel-softmax/Gumbel-sigmoid/log-normal draws so gradients can flow, but reporting $\hat{W}_t$ from a single such draw carries irreducible sampling variance even once (η, λ, q) are well-calibrated — this showed up empirically as a flat/noisy `model_rel_error` in the first synthetic run while the reconstruction likelihood term improved steadily underneath it. At inference/evaluation time, $\hat{W}_t$ should instead be built from the posterior-predictive **mean** at each stage: $q_\psi(z)$ itself (no Gumbel-softmax draw), $\sigma(\eta)$ for existence (no Gumbel-sigmoid draw), and $\lambda \cdot e^{\sigma^2/2}$ for magnitude (the log-normal mean, no reparameterized draw) — then project through Algorithm 1 as usual. This is a deliberate design decision, not just an experiment-script detail: it governs how both the synthetic validation (§12) and the real-data baseline comparison (§7) numbers get computed.

1.8 7. Design rationale

- **SBM over preferential-attachment/fitness models** as the sole structural backbone: core-periphery tiering in interbank markets is literally a latent-block story, so SBM is the natural fit. PA/fitness models (Caldarelli et al.) are cited in related work for contrast but not built.
- **GRU, not a temporal-GNN** (EvolveGCN, TGN, etc.) for the temporal mechanism: deliberately the simpler choice, given the "big model, small data" risk — roughly 50–80 institutions, at most ~170 periods. A fancier temporal-GNN would be over-parameterized for this panel size.
- **Mandatory baseline**: head-to-head comparison against Gandy–Veraart / RAS maximum-entropy network reconstruction, on both the synthetic SBM data (known ground truth) and the real panel. Explicit instruction from the design discussion: report honestly if the deep model doesn't beat the classical baseline — don't bury a negative result.

1.9 8. Variable inventory (what actually feeds the model)

Net statement from the design discussion: **the reconstruction model needs ~5 covariates per node per period, plus the two interbank marginals. Everything else in the NRB bulletins is either a Paper 2 outcome variable or not usable given current data coverage.**

1.9.1 8a. The two marginals (non-negotiable — the reconstruction target)

The names below (`2.b Interbank Borrowing`, `INTERBANK LENDING (Marginal r_i)`) were this doc's own shorthand, not literal bulletin text, and the row-marginal name additionally implied a standalone top-level item that doesn't exist. Confirmed against the real extracted CSV (`nrb_balance_sheet.csv`) on 2026-10-22 — see §9.4:

Symbol	Real bulletin text	Parent section	Statement / side
$c_{i,t}$ (column marginal)	`b. Interbank Borrowing`	`2 BORROWINGS`	Balance Sheet, **Liabilities**
$r_{i,t}$ (row marginal)	`a. Interbank Lending`	`3 SHARE & OTHER INVESTMENT`	Balance Sheet, **Assets**

Both are lettered sub-items, not top-level rows — `r_{i,t}$` in particular is a breakdown component of `3 SHARE & OTHER INVESTMENT`'s own reported total, which matters for the size formula below (§9.4). These are the actual payoff variables for the whole method — everything else exists to help infer the matrix whose row/column sums these fix.

1.9.2 8b. Node covariates $X_{i,t}$ (block-membership / fitness prior)

#	Covariate	Built from	Role
1	Size	sum of the **10** real top-level asset rows: `1 LIQUID FUNDS`, `2 INVESTMENT IN SECURITIES`, `3 SHARE & OTHER INVESTMENT`, `4 LOANS & ADVANCES`, `5 BILL PURCHASED`, `6 LOANS AGAINST COLLECTED BILLS`, `7 FIXED ASSETS`, `8 OTHER ASSETS`, `9 EXPENSES NOT WRITTEN OFF`, `10 NON BANKING ASSETS` (**not** the `TOTAL ASSETS` label directly — see §9.1). $r_{i,t}$ is **not** added as an 11th term — per §8a/§9.4 it's a sub-item already folded into `3 SHARE & OTHER INVESTMENT`'s reported total, so adding it separately would double-count it	SBM block/fitness prior, scale control
2	Capital ratio	`1 CAPITAL FUND` (Liabilities) ÷ size	solvency/fitness term; also sets the obligor cap $K_{i,t}$
3	Liquidity ratio	`1 LIQUID FUNDS` (Assets) ÷ size	liquidity position
4	Loan-to-deposit pressure	`4 LOANS & ADVANCES` (Assets) ÷ `3 DEPOSITS` (Liabilities)	funding-pressure signal

#	Covariate	Built from	Role
5	NRB-dependence	`a. NRB` ÷ `2 BORROWINGS` (both under Liabilities, `a. NRB` nested under `2 BORROWINGS`; if `2 BORROWINGS` is missing for a row — some early-era filings never print the top-level subtotal — the denominator is reconstructed as the sum of its sub-items: `a. NRB`, `b. Interbank Borrowing`, `c. Foreign Banks and Fin. Ins.`, `d. Other Financial Ins.`, `e. Bonds and Securities` — confirmed real text per §9.4; the `f./g.` sub-items this doc previously assumed existed under `2 BORROWINGS` have **not** been confirmed in the real data yet)	central-bank liquidity dependence — useful for the zero-inflation gate (a bank funded mainly by NRB is less likely to have interbank exposure)

Optional secondary signal, not in the core 5: ownership type (state/private/foreign), if available — would add homophily structure to the block prior.

1.9.3 8c. Auxiliary observed-graph candidate (optional GNN refinement input, §4)

Source	Role
Statement of Credit, sector-wise breakdown (12 sectors + Others)	shared sector-concentration = a correlated-shock signal independent of the interbank marginals. Candidate as the auxiliary *observed* graph the GNN layer can refine over, since it doesn't require W_t itself.

1.9.4 8d. Explicitly excluded from Paper 1

- `6 Bills Payable`, `5 Other Liabilities` (Liabilities, minor residuals) — not used
- `6/10 Reconciliation A/c` (both sides) — accounting plug, not economically meaningful
- `8 Expenses not Written off`, `9 Non Banking Assets` (Assets) — legacy/minor, not used
- Everything in Profit & Loss — these are Paper 2 outcome variables, not reconstruction inputs (see `paper2_causal_spillover.md`)
- `nrb_statement_of_credit.csv` as a panel source — only 2 years of coverage in current data, can't support a 14-year panel; not worth cleaning further unless more years turn up uncollected

1.9.5 8e. One-to-one equation-symbol raw-variable map

Every symbol that appears in Eq. (1)–(8) is listed below with exactly what real NRB field (if any) it is computed from. Symbols with no raw field are latent or learned — they're estimated by the model, not read off the bulletin.

Symbol	Appears in	Raw NRB field	Statement / side	Transformation
$r_{i,t}$	Eq. (5), (6), (7), (8)	`a. Interbank Lending` (sub-item of `3 SHARE & OTHER INVESTMENT`)	Balance Sheet, Assets	identity (reported value)
$c_{i,t}$	Eq. (5), (6), (7), (8)	`b. Interbank Borrowing` (sub-item of `2 BORROWINGS`)	Balance Sheet, Liabilities	identity (reported value)
$K_{i,t}$	Eq. (8), $\kappa K_{i,t}$ cap	`1 CAPITAL FUND`	Balance Sheet, Liabilities	identity (reported value) — note: raw value here, *not* the ratio used in $X^{(2)}_{i,t}$ below

Symbol	Appears in	Raw NRB field	Statement / side	Transformation
$X_{i,t}^{(1)}$	Eq. (2), component of $X_{i,t}$	sum of the 10 real top-level asset rows (listed in §8b)	Balance Sheet, Assets	sum (not the `TOTAL ASSETS` label — see §9.1; $r_{i,t}$ excluded, see §8b/§9.3)
$X_{i,t}^{(2)}$	Eq. (2)	`1 CAPITAL FUND`	Balance Sheet, Liabilities	$\div X_{i,t}^{(1)}$ (capital ratio)
$X_{i,t}^{(3)}$	Eq. (2)	`1 LIQUID FUNDS`	Balance Sheet, Assets	$\div X_{i,t}^{(1)}$ (liquidity ratio)
$X_{i,t}^{(4)}$	Eq. (2)	`4 LOANS & ADVANCES ÷ 3 DEPOSITS`	Balance Sheet, Assets ÷ Liabilities	ratio across statement sides
$X_{i,t}^{(5)}$	Eq. (2)	`a. NRB ÷ 2 BORROWINGS`	Balance Sheet, Liabilities (both)	ratio within Liabilities; denominator falls back to the sum of `a.`–`e.` sub-items (§8b) when `2 BORROWINGS` is unreported
w	Eq. (2)	—	—	**learned** (fitness weight vector, dim = 5, no raw field)
z_i	Eq. (1), (3), (7)	—	—	**latent** (inferred via q_ψ, Eq. 7); optionally informed by ownership type if used (§8b)
Λ	Eq. (3)	—	—	**learned** ($K \times K$ block-pair intensity matrix)

Symbol	Appears in	Raw NRB field	Statement / side	Transformation
$h_{i,t}$	Eq. (3), (6), (7)	$X_{i,t}^{(1\text{-}5)}$ (all 5 covariates) and the observed marginals $r_{i,t}, c_{i,t}$, concatenated as the GRU input vector	—	**learned embedding**, GRU output — not read directly off any single field
θ (GRU weights)	Eq. (6)	—	—	**learned**
ψ (posterior-net weights)	Eq. (7)	—	—	**learned**
$\eta_{ij,t}$	Eq. (4)	—	—	**learned** (zero-inflation logit, decoder output)
σ^2	Eq. (4)	—	—	**learned** (log-normal dispersion — global or per-block, §10)
κ	Eq. (8)	—	—	**exogenous regulatory constant** (single-obligor exposure limit; not reported in the bulletin itself — a known regulatory parameter)
A^{obs} (auxiliary GNN graph, §4)	Eq. (6) refinement, optional	sector-wise credit, 12 sectors	Statement of Credit	co-exposure adjacency built from shared-sector lending shares

Reading guide: $X_{i,t} = (X_{i,t}^{(1)}, ..., X_{i,t}^{(5)}) \in \mathbb{R}^5$ is the concrete instantiation of the generic $X_{i,t} \in \mathbb{R}^d$ in §2 — i.e. $d = 5$ for this model, and the ordering above (size, capital ratio, liquidity ratio, LDR, NRB-dependence) is the fixed component order used everywhere $X_{i,t}$ appears in Eq. (2) and (6).

1.10 9. Known data-pipeline caveats (apply before this goes into the model)

1. **`item_code` collision**: confirmed by re-running `nrb_extractor.py` against a real bulletin — `item_code` is not unique across the Balance Sheet's Liabilities and Assets sides. Both sides independently restart numbering at 1 (e.g. `1 CAPITAL FUND` on Liabilities vs. `1 LIQUID FUNDS` on Assets; same collision pattern for `2`, `3`, and lettered sub-items `a.`–`g.`). The extractor (`nrb_extractor.py:197-269`, `_extract_rows`) currently has no column marking which side a row is on — that currently has to be inferred externally from which `TOTAL` block (`TOTAL Liabilities` vs. `TOTAL ASSETS`) a row falls between on the page. **Fix needed**: add a `side` column to the extractor output before any join on `item_code` alone, or it will silently collide liabilities and assets rows that share a code (e.g. covariates 2 and 3 above, which both use an `item_code == 1` row from different sides).
2. Rotated bank-name header recovery (`recover_bank_names`, `nrb_extractor.py:130-171`) and the layout-mode row extraction are both working as validated against the real sample PDF — no open bug there as of this pass.
3. **`item_code` drift across eras**: the top-level code attached to a given labeled row is not stable over the panel's history — e.g. `LOANS & ADVANCES` is coded `4` in some eras and `4.1` in others (with `BILL PURCHASED` shifting from `5` to `4.2` correspondingly). Matching must key off the normalized label text (and, for lettered sub-items, parent section), never off `item_code` alone.
4. **Lettered sub-items (`a.`–`g.`) are reused across unrelated parent sections** (e.g. `a.` means `Paid-up Capital` under `1 CAPITAL FUND`, `Cash Balance` under `1 LIQUID FUNDS`, `NRB` under `2 BORROWINGS`, and `Interbank Lending` under `3 SHARE & OTHER INVESTMENT` — all in the same bulletin). The letter alone is not a key; a row's identity is (normalized label text, nearest preceding top-level/bare-number row in extraction order). The extractor should carry this parent link explicitly (a `parent_item` or `section` column) rather than requiring every consumer to replay the state machine itself. **Side can differ across the same reused label, too**: `e.` `Others` appears under both `3 DEPOSITS` (Liabilities) and `8 OTHER ASSETS` (Assets) in the same bulletin (confirmed `Asar-2074.pdf`/NBL, 2026-10-22). Side-consistency checks therefore must only be enforced for the small set of labels the model actually reads (§8b/§8e's 8 fields) — requiring every label in the bulletin to resolve to one consistent side will raise on this kind of legitimate, unrelated collision.
5. **Truncated-text duplicate rows**: some extractions clip a label mid-word (e.g. `4 LOANS &` alongside the correct `4 LOANS & ADVANCES`, or `1 CAPITAL` alongside `1 CAPITAL FUND`) for the same `item_code`/concept. Confirmed via `cut -d, -f7,8 ... | sort -u` on the real CSV (2026-10-22). Treat these as the same field (prefix match or longest-label-wins), not as missing data for the truncated variant.
6. **Pre-2075-era bulletins never extract a `TOTAL Liabilities` marker row at all** (confirmed against real `Asar-2074.pdf`, 2026-10-22) — the Liabilities section just ends and the Assets section starts back at `item_code "1"` with no boundary row to key off. Side inference needs a fallback for this era that doesn't depend on the marker existing: a drop back to a lower top-level bare-digit `item_code` while still in the Liabilities state is itself the side boundary (top-level codes strictly

increase within one side).

7. **Real bulletins extract row-major by line item, interleaved across banks** — bank A's row for item N is immediately followed by bank B's row for item N, not bank A's entire sheet followed by bank B's (confirmed 2026-10-22; caused a live "inconsistent side" error before this was known). Side inference must replay its liabilities/assets state machine **per `(source_file, bfi_name)` bank-block**, re-sorted by original extraction order — not as one shared state machine over the raw file order — or one bank's `TOTAL LIABILITIES` marker flips the state for every other bank's very next row too.
8. **Blank value cells are dropped from extraction entirely, not reported as a zero.** A row with an empty value field (seen on optional/granular categories some banks don't report, e.g. `9 EXPENSES NOT WRITTEN OFF`) is simply absent from that bank/period's record, not present with value 0. Any field built by summing a fixed list of categories (e.g. the `size` covariate, §8b) must treat a missing category as 0 explicitly rather than looking it up strictly — a strict lookup wrongly drops the entire bank/period cell over one legitimately-absent optional row.

1.11 10. Open items (not yet decided)

- K **(number of SBM blocks)**: start with $K = 2$ (core/periphery), treat larger K as a robustness check, not the headline specification.
- σ^2 **(log-normal dispersion)**: learned, either globally or per-block — not yet decided which.
- **Venue/scope hook**: ICAIF fit is comfortable at the current scope (methods + real-data case study). ICLR/NeurIPS main track would need an added theoretical hook — e.g. a provable statement on how reconstruction uncertainty in $\hat{W}_t$ propagates into coverage guarantees for a downstream interference estimator (which would also strengthen the Paper 1 → Paper 2 link). **Not resolved** — needs a decision before drafting.
- **Posterior collapse on the block-assignment KL term — root cause identified, architecture fix plus capped KL ceiling fixes reconstruction but NOT block recovery reliably across seeds**: Eq. 9's $\mathrm{KL}(q_\psi(z)\|p(z))$ under the original unweighted ($\beta_t = 1$ always) term collapsed to ~0 almost immediately in the first synthetic validation run. A direct check (`argmax(q)` vs. the synthetic ground-truth z, best accuracy over label permutations since block labels are unidentifiable) confirmed this is real collapse, not just a small-but-harmless KL value: block-recovery accuracy *fell below* the trivial "always guess the majority block" baseline (`max(pi)`, not $1/K$, since `pi` is imbalanced) as training proceeded. This persisted even after §11 Algorithm 2's existence-gate fix (giving η a block-dependent structure via `eta_block`, mirroring Λ) and after β_t KL annealing (linear warm-up, §6, `TrainStepConfig.kl_beta`): annealing stopped block-recovery accuracy from falling *below* the trivial baseline, but it only ever plateaued exactly at that baseline, never above it. Root cause: $q_\psi(z_i)$, prior to the Eq. 6-7 fix, was conditioned on $X_{i,1:t}$ alone — but the generative model draws z_i (Eq. 1) and X_i (Eq. 2) independently, so the encoder had *zero* information from which to ever recover z above the trivial majority-class floor, regardless of how the loss is reweighted. Fix: the encoder's input was widened to $[X_{i,t}; r_{i,t}; c_{i,t}]$ — the observed

interbank lending/borrowing marginals, which *are* functions of z through $\Lambda[z_i, z_j]$ — matching §4's own already-stated (but previously unimplemented) intent of $q_\psi(W_t \mid X_{1:t}, r_t, c_t)$. The first re-run with it in place showed block-recovery accuracy *peak at 0.85* — well above the trivial baseline — confirming r_t, c_t do carry real block signal once the encoder can see them, but accuracy then decayed back toward/below the trivial baseline as β_t annealed toward 1: annealing to a full $\beta_t = 1$ reproduces the original collapse on a delay, this time by out-competing a signal that *does* exist rather than one that never did. Mitigation: cap β_t's asymptote at $\beta_{\max} = 0.15 < 1$ instead of annealing to 1 (§6). A single 3000-epoch re-run with this cap (seed 0) held block-recovery accuracy at 0.7578–0.7682 for the entire run (epoch 200 through 2999), against that run's trivial baseline of 0.7344 — KL stabilized around 266 rather than collapsing, and $\hat{W}_t$'s relative Frobenius error kept improving in parallel (to 0.4297, vs. baseline 0.5965). **A 5-seed replication (seeds 0–4) showed this does NOT generalize**: block-recovery margin over the trivial baseline was $+0.0234, -0.1380, +0.1380, -0.1797, -0.0026$ across seeds 0–4 — mean -0.0318, std 0.1148 (std nearly 4× the mean) — with only 2/5 seeds beating the trivial baseline at all. Seed 0's healthy-looking run was not representative; on average across seeds, block recovery is statistically indistinguishable from (if anything slightly worse than) the trivial majority-guess baseline. Two distinct failure modes recur: full collapse to the uniform prior (KL ≈ 0 throughout — seed 4), and confident-but-wrong block assignment (KL climbs to a healthy magnitude but accuracy sits *below* the trivial baseline anyway — seed 1). Reconstruction accuracy, in contrast, **is** robust across the same 5 seeds: model mean relative Frobenius error beat the classical Sinkhorn/RAS baseline in 5/5 seeds (mean 0.4154 vs. 0.5657, std 0.0416 vs. 0.0257). **Resolved conclusion for now**: the paper's claim that "the model identifies which banks are core" is **not currently supported** by this architecture + hyperparameter combination — it should either

(a) be dropped in favor of the already-robust claim ("the model recovers W_t more accurately than the classical baseline," no block-identity claim), or (b) remain an explicit goal pending further work: longer training, more `BlockPosteriorMLP` capacity, a lower $\beta_{\max}$, or a per-seed diagnosis of why seeds 1/4 fail differently from seeds 0/2. Not yet decided which path; revisit once real-data validation (§9/§12) is underway, since that may be a better use of time than chasing a synthetic-only metric further.

1.12 11. Algorithms (pseudocode)

Three algorithms, in dependency order: the capacitated projection (the hard constraint layer, Eq. 8) is standalone and testable without the rest of the model; the generative forward pass (Eq. 1-5, 8) samples a synthetic W_t for validation; training (Eq. 6-9) wraps the amortized encoder/decoder of §4 around the same projection layer. All three are written over *symbol-level* inputs (N, K, d, κ, etc.) — nothing below is a fixed dataset size or constant; see §12 for how these get threaded through as config, not literals.

1.12.1 Algorithm 1 — `CapacitatedProjection` (Eq. 8, Dykstra's alternating-projections algorithm)

```
Input:  W̄   R^{N×N}_{ 0}      raw/sampled edge weights
        r, c   R^N_{ 0}        target row/column marginals (observed r_i,t, c_i,t)
        cap   R^N_{ 0}         per-row obligor cap, cap_i =   · K_i,t
        n_iters, tol           stopping criteria (n_iters counts full row→col→box
                                passes, not single-set applications)
Output: W   R^{N×N}            satisfies row/col marginals and W_ij   cap_i for all j,
                                exactly (up to tol), when the intersection is non-empty
```

```
 1:  W ← W̄
 2:  I_row, I_col, I_box ← zeros(N,N), zeros(N,N), zeros(N,N)   # Dykstra correction terms
 3:  for iter = 1 .. n_iters:
 4:      W_prev ← W
 5:      Y ← W + I_row
 6:      W ← Y + (r − row_sums(Y)) / N          # Proj onto {row sums = r}, broadcast per row
 7:      I_row ← Y − W
 8:      Y ← W + I_col
 9:      W ← Y + (c − col_sums(Y)) / N          # Proj onto {col sums = c}, broadcast per col
10:      I_col ← Y − W
11:      Y ← W + I_box
12:      W ← clip(Y, 0, cap)                     # Proj onto {0   W_ij   cap_i}, per-row cap,
                                                 #   applied to EVERY entry in row i
13:      I_box ← Y − W
14:      if ‖W − W_prev‖ < tol: break
15: return W
```

Note: unlike the superseded Sinkhorn-clip draft (see §5's revision note), this provably converges to the *exact* Euclidean projection onto the intersection of all three constraint sets whenever that intersection is non-empty (Boyle & Dykstra, 1986) — not a heuristic. All three per-set projections (lines 6, 9, 12) are closed-form and differentiable (the clip has the usual subgradient at the boundary, same as a ReLU), so gradients flow through to $\tilde{W}_t$ exactly as Eq. 8 requires for Algorithm 3's step 10. If the intersection is empty — e.g. $\kappa K_{i,t}$ set so tight that $N \cdot \kappa K_{i,t} < r_{i,t}$ for some i, meaning the cap can't physically supply the required row total even by maxing out every entry — this converges to a least-violation compromise instead, which should be treated as an infeasible-cap data/parameter error, not silently accepted (`test_capacity_projection.py`'s `infeasible_case` checks for this — the cap itself must still hold even when the row marginal can't be reached).

1.12.2 Algorithm 2 — `SampleSyntheticNetwork` (Eq. 1-5, 8; ground truth for validation, §6)

```
Input:  N, K, d                  problem size (num banks, num blocks, covariate dim)
        Δ^K, Λ   R^{K×K}_{ 0}, H   R^{K×K}, w   R^d,  ²        generative params (drawn
                                                               from a prior or fixed per synthetic
                                                               scenario); H is the block-pair
                                                               zero-inflation logit matrix (Eq. 4),
                                                               block-structured the same way Λ is
                                                               (Eq. 3) - existence, not just
                                                               magnitude, needs a block-dependent
                                                               pattern for q_psi(z) to have any
                                                               recon-loss signal to recover it from
        X_{1:N}   R^{N×d}        covariates for this period (synthetic or real)
         , K_cap   R^N           regulatory cap inputs
Output: W_true   R^{N×N}_{ 0}, r, c   R^N      ground-truth network + its implied marginals

1:  for i = 1 .. N: z_i ~ Categorical( )                    # Eq. 1
2:  for i = 1 .. N:  _i ← exp(w^T X_i)                      # Eq. 2
3:  for i, j = 1 .. N (i   j):
4:       _ij ← Λ[z_i, z_j] ·  _i ·  _j                      # Eq. 3 (no h_i·h_j term -
no
                                                              #   history yet for a one-shot sample
5:       _ij ← H[z_i, z_j]; e_ij ~ Bernoulli( ( _ij))       # Eq. 4, block-structured same as step
6:      W_ij ← e_ij · LogNormal_sample(log  _ij,  ²)
7:  r ← row_sums(W); c ← col_sums(W)                        # pre-projection marginals
8:  W_true ← CapacitatedProjection(W, r, c,  ·K_cap)        # Eq. 8 - ground truth MUST
                                                              #   also satisfy the cap, or the
                                                              #   model is validated against an
                                                              #   infeasible target
9:  return W_true, row_sums(W_true), col_sums(W_true)
```

1.12.3 Algorithm 3 — `TrainStep` (Eq. 6-9; amortized inference + one optimizer step)

```
Input:  batch of periods {X_{1:N,t}, r_t, c_t, K_t}_{t   batch}, optional W_true_t (synthetic only)
        model params   (GRU),   (block/decoder nets), Λ, w,  ²
        hyperparams  _smooth,  _reg,
State:  h_{i,t-1} carried across periods (GRU hidden state)
Output: loss value; updated  ,  , Λ, w after one gradient step
```

```
for t in batch (in temporal order):
    for i = 1 .. N: h_{i,t} ← GRU_ ([X_{i,t}; r_{i,t}; c_{i,t}], h_{i,t-1})  # Eq. 6
    [optional] h_{i,t} ← GNN_refine(h_{i,t}, A_obs_t)                  # §4, §8c
    for i = 1 .. N: q_i ← softmax(MLP_ (h_{i,t}))                     # Eq. 7
    for i = 1 .. N: z_i ← GumbelSoftmaxSample(q_i)                     # reparameterized
    for i, j = 1 .. N (i   j):
        ( _ij,t,  _ij,t) ← Decoder_ (h_i,t, h_j,t, z_i, z_j)          # Eq. 3-4
        e_ij,t ← GumbelSigmoidSample( _ij,t)                           # reparameterized gate
        W_ij,t ← e_ij,t · LogNormalSample(log  _ij,t,  ²)             # reparameterized weight
    W_t ← CapacitatedProjection(W_t, r_t, c_t,  ·K_t)          # Eq. 8, Algorithm 1
    if W_true_t is given (synthetic branch):
        L_recon,t ← -log p(W_true_t |  _t,  _t,  ²)                    # Eq. 9, first term
    else:
        L_recon,t ← 0                                                  # real data: no ground tru
    L_KL,t ← KL(q_ (z_t) ‖ p(z))
L_smooth ← Σ_t ‖W_t - W_{t-1}‖_F²            over consecutive periods in the batch
L_reg ← ‖Λ‖² + ‖w‖²
L ← Σ_t (L_recon,t + L_KL,t) +  _smooth · L_smooth +  _reg · L_reg
backprop L; update  ,  , Λ, w via optimizer step
return L
```

1.13 12. Paper structure (7 pages)

Section	Length	Content
Intro + related work	~1.5p	position against Aronow-Samii/Leung (network assumed *known*), Elliott-Golub-Jackson (network assumed known, no reconstruction), Gandy-Veraart (reconstruction without covariate-informed priors or regulatory constraints)
Method	~2.5p	§3–6 above
Synthetic validation	~1.5p	SBM-generated ground truth, baseline comparison
Real-data case study	~1p	Nepal panel, temporal-stability evidence

Section	Length	Content
Discussion/limitations	~0.5p	honest treatment of the no-ground-truth-on-real-data limitation